Você é meu mentor de implantação do PortfolioHUB, meu portfólio no GitHub (repositório Meu_Study_Hub_TI + GitHub Pages). Sou estudante de Engenharia de Software. Guie-me passo a passo em 6 fases: (1) planejamen

Com certeza! Vamos estruturar o seu **PortfolioHUB** (repositório `Meu_Study_Hub_TI`) como um verdadeiro ecossistema profissional de documentação, projetos e portfólio no GitHub Pages. Como estudante de Engenharia de Software, aplicar boas práticas de arquitetura, segurança e versionamento desde o início fará toda a diferença no seu perfil.

Fase 1: Planejamento e Estruturação

Nesta fase, definimos a arquitetura de arquivos e a utilidade de cada pasta para evitar repositórios desorganizados ("bagunçados") no futuro.

Checklist

- [ ] Definir a árvore de diretórios do repositório.
- [ ] Criar o arquivo `README.md` principal (a "vitrine" do repositório).
- [ ] Mapear quais arquivos serão estáticos (HTML/CSS/JS) para publicação no GitHub Pages.
- [ ] Criar o arquivo `.gitignore` logo no primeiro `git init`.

Por que esta fase é crucial?

Um repositório sem planejamento prévio tende a misturar anotações pessoais, códigos de teste e documentação pública. Estruturar a arquitetura de informação facilita a navegação do recrutador e a manutenção do seu material de estudo.

Exemplo de Estrutura Recomendada

Plaintext

```
Meu_Study_Hub_TI/
├── index.html              # Página principal do GitHub Pages
├── assets/                 # Imagens, CSS, scripts do site
│   ├── css/
│   └── img/
├── disciplinas/            # Anotações organizadas por matéria
│   ├── engenharia-software/
│   └── estrutura-dados/
├── projetos/               # Submódulos ou links para outros projetos
├── .gitignore              # Proteção contra subida de lixo/segredos
└── README.md               # Apresentação do repositório
```

Boas Práticas de Segurança

Proteção do `.gitignore`: Inclua arquivos temporários de IDEs (`.vscode/` , `.idea/`), arquivos de ambiente (`.env`), logs (`*.log`) e dependências (`node_modules/`).

Navegação limpa: Nunca armazene rascunhos com dados pessoais sensíveis (como telefone pessoal, senhas ou endereços reais) em pastas do projeto.

Fase 2: Integração e Versionamento com Git & GitHub

Aqui estabelecemos o fluxo de trabalho profissional usando versionamento semântico e publicamos o site via GitHub Pages.

① Inicializar o Repositório Local e Conectar ao GitHub

Abra o terminal na pasta do projeto e execute os comandos para inicializar o repositório, criar a branch principal `main` e vincular ao repositório remoto.

Bash

```
git init
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/Meu_Study_Hub_TI.
```

② Fazer o Primeiro Commit das Estruturas

Adicione os arquivos organizados na Fase 1 e faça o commit inicial utilizando mensagens claras.

Bash

```
git add .
git commit -m "feat: estrutura inicial do repositorio e pagina princip
git push -u origin main
```

③ **Configurar a Publicação no GitHub Pages**

No repositório do GitHub, acesse **Settings** > **Pages**. Em **Build and deployment**, selecione a branch `main` e a pasta `/ (root)`. Clique em **Save**. Seu portfólio estará acessível em `https://SEU_USUARIO.github.io/Meu_Study_Hub_TI/`.

Por que esta fase é crucial?

O uso correto do Git demonstra maturidade técnica. Mensagens de commit padronizadas (ex: *Conventional Commits*) mostram a recrutadores que você já entende a rotina de desenvolvimento de um time de software.

Boas Práticas de Segurança

Uso de Chaves SSH ou Tokens PAT: Não utilize senhas diretas no terminal. Configure chaves SSH com senha (*passphrase*) para autenticação segura com o GitHub.

Evitar o Commit de Credenciais: Nunca suba senhas, tokens de API ou chaves privadas. Caso envie algo sem querer, trocar a credencial imediatamente é a única medida 100% segura (não basta apenas apagar o commit do histórico).

Fase 3: Gestão de Usuários, Colaboração e Segurança

Garantir quem pode alterar o código e como as atualizações chegam à branch principal.

Checklist

[ ] Ativar proteção de branch na `main` (*Branch Protection Rules*).

[ ] Definir o modelo de colaboração (Pull Requests em vez de push direto).

[ ] Configurar varredura de segredos (*Secret Scanning*).

Por que esta fase é crucial?

Mesmo em um projeto individual, simular o ambiente corporativo com regras de proteção impede substituições acidentais do histórico do código (`git push --force`) e garante integridade ao publicar atualizações.

Regras de Proteção recomendadas no GitHub

Em **Settings** > **Branches** > **Add branch protection rule**:

1. Digite `main` no padrão de nome.
2. Marque **Require a pull request before merging**.
3. (Opcional) Marque **Require status checks to pass before merging** se configurar automações de teste.

Boas Práticas de Segurança

Princípio do Menor Privilégio: Caso adicione colegas de faculdade para colaborar, atribua papéis adequados (ex: *Write* para colaboradores, *Admin* apenas para você).

Habilitar Secret Scanning e Dependabot: Em **Settings** > **Code security and analysis**, ative o *Secret Scanning* e o *Dependabot alert* para ser avisado se alguma vulnerabilidade for detectada nas suas dependências.

Fase 4: Compartilhamento e Controle de Acesso

Definição da visibilidade do repositório, licença de uso e apresentação executiva para a comunidade e recrutadores.

Checklist

[ ] Escolher uma licença open-source (ex: MIT License).

[ ] Adicionar o link do GitHub Pages na seção **About** do repositório.

[ ] Criar templates para *Issues* e *Pull Requests* (opcional para padrão corporativo).

[ ] Incluir badges de status e tecnologias no `README.md` .

Por que esta fase é crucial?

Um repositório sem licença explícita é, por padrão, protegido por direitos autorais estritos, o que impede reutilizações por outros estudantes. Além disso, um `README.md` atraente com link direto para a página publicada é o que retém a atenção de quem visita seu perfil.

Estrutura Sugerida para o `README.md`

Markdown

```
# 🚀 PortfolioHUB | Meu Study Hub de TI

Bem-vindo ao meu hub de estudos e portfólio de Engenharia de Software!

🔗 **Acesse o site ao vivo:** [Meu_Study_Hub_TI](https://SEU_USUARIO.github

## 🛠️ Tecnologias
- HTML5 / CSS3 / JavaScript
- Git & GitHub Pages

## 📂 Organização
- `/disciplinas`: Resumos e mapas mentais das matérias.
- `/projetos`: Projetos práticos desenvolvidos durante a graduação.

## 📄 Licença
Este projeto está sob a licença [MIT](LICENSE).
```

Boas Práticas de Segurança

Não expor informações privadas: Verifique se resumos acadêmicos não contêm dados de professores, colegas ou trabalhos com termos de confidencialidade (*NDA*) de estágios/empresas.

Fase 5: Testes, Automação e Deploy Contínuo (CI/CD)

Validar se o site está funcionando corretamente, se os links não estão quebrados e automatizar verificações.

Checklist

[ ] Validar links internos e externos no GitHub Pages.

[ ] Testar a responsividade do site (mobile e desktop).

[ ] (Avançado) Criar um fluxo de validação com **GitHub Actions**.

Por que esta fase é crucial?

Links quebrados e páginas com erros visíveis causam má impressão profissional. Automatizar verificações garante que cada atualização na branch `main` mantenha o nível de qualidade.

Exemplo de Workflow com GitHub Actions (Linter / Verificação)

Crie o arquivo `.github/workflows/lint.yml` :

YAML

```yaml
name: Check HTML & Links

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Validate HTML
        run: |
          echo "Executando verificações de qualidade e estrutura..."
```

Boas Práticas de Segurança

Integridade de Recursos Externos (SRI): Ao utilizar bibliotecas ou estilos via CDN (como Bootstrap ou FontAwesome), utilize sempre as tags com os atributos `integrity` e `crossorigin` para garantir que o arquivo externo não foi adulterado.

HTML

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/c
```

Fase 6: Revisão Final e Auditoria

Antes de divulgar o portfólio no seu LinkedIn ou currículo, faça um Pente Fino técnico.

Checklist de Entrega

Item	Descrição	Status
Página no Ar	O link do GitHub Pages carrega sem erros 404?	[ ]
Sem Segredos	O histórico do Git está limpo de senhas/tokens?	[ ]
Navegação	O `README.md` orienta com clareza qualquer visitante?	[ ]
Responsividade	O layout funciona bem em telas de celular?	[ ]
Licença	O repositório contém o arquivo `LICENSE`?	[ ]

Por que esta fase é crucial?

A revisão final é a garantia de qualidade (*Quality Assurance*) do seu próprio produto. Ela impede que pequenos descuidos comprometam o impacto do seu trabalho perante recrutadores.

Boas Práticas de Segurança e Manutenção Contínua

Histórico de Commits: Verifique se o seu e-mail configurado no Git local (